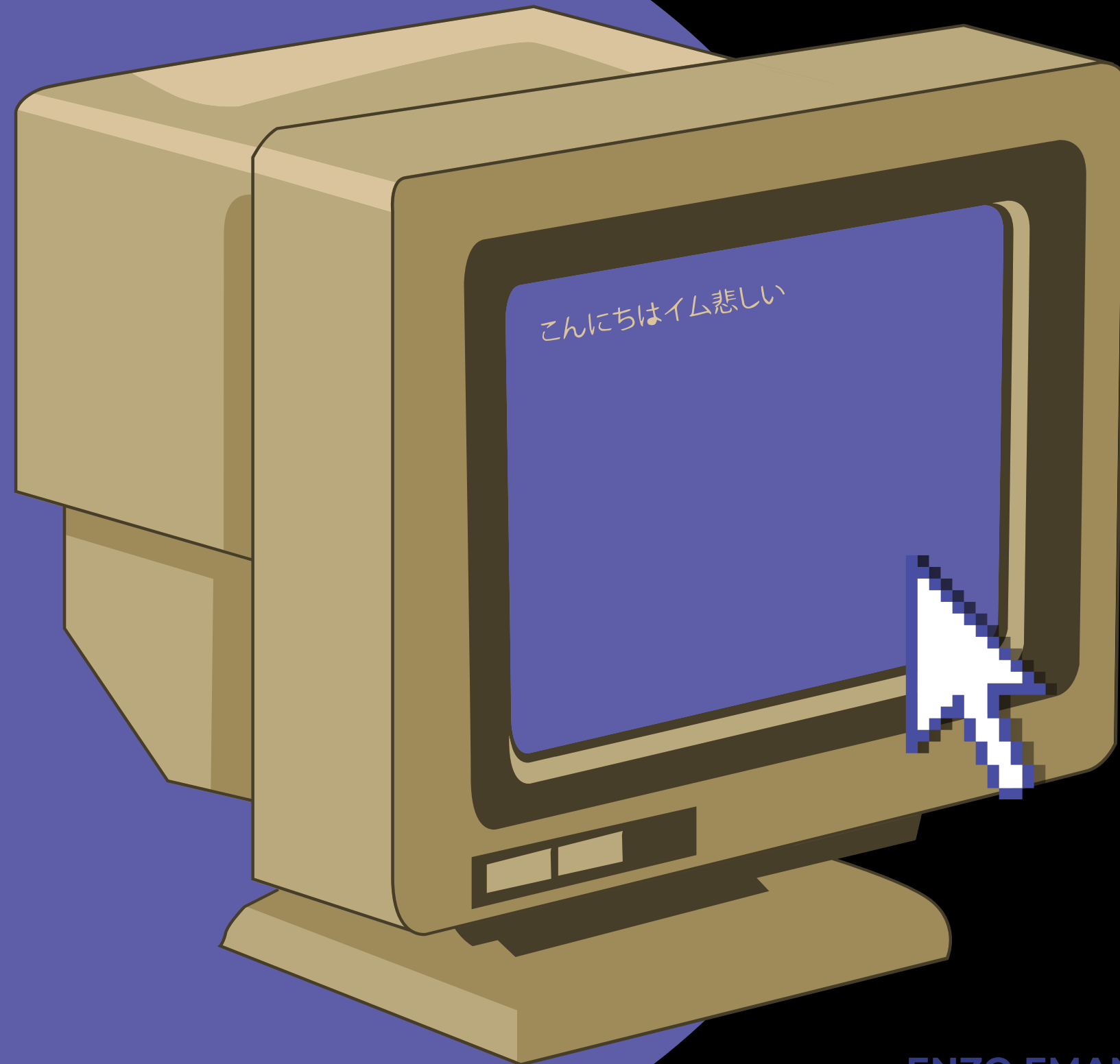




SISTEMAS OPERACIONAIS

EQUIPE GENMAN: TRILHA A — CHAT
LOCAL COM OLLAMA + OPEN WEBUI

ENZO EMANUEL COSTA MAIA, FILIPE DE CARVALHO GODOY, GIULIAN FÁBIO
BASTOS AMORIM LIMA, JOSÉ FAGNER SILVA JUNQUEIRA, KATYANE DOS SANTOS,
RAUANY INGRID SANTOS DE JESUS E VÊNISON CARDOSO DOS SANTOS

AMBIENTE EXPERIMENTAL

No ambiente WSL2 utilizado como referência para o inventário, foram identificados 4 núcleos físicos e 8 threads, utilizando um processador Intel Core i5-8265U, aproximadamente 6,1 GiB de memória RAM e 2 GiB de memória swap. Durante a inicialização do Ollama, o runtime identificou a execução utilizando CPU, com 0 B de VRAM disponível para o processo de inferência.

```
zoz@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ nproc
8

zoz@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ free -h
              total        used        free      shared  buff/cache   available
Mem:           6.1Gi        234Mi        5.9Gi          68K       1.9Gi        5.7Gi
Swap:          2.0Gi           0B          2.0Gi           0B

zoz@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ df -h
Filesystem      Size  Used Avail Use% Mounted on
/dev/sdb1       251G   4.3G   234G   2% /
efusefs         3.1G     0   3.1G   0% /mnt/wsl
cgroupfs        222G  176G   46G   80% /init
/dev/loop0      3.1G   4.0K   3.1G   1% /run
/dev/loop1      3.1G     0   3.1G   0% /run/lock
/dev/loop2      3.1G     0   3.1G   0% /run/shm
/dev/loop3      3.1G     0   3.1G   0% /run/user
/dev/loop4      3.1G     0   3.1G   0% /sys/fs/cgroup
/dev/loop5      222G  176G   46G   80% /usr/lib/wsl/dri
/dev/loop6      222G  176G   46G   80% /usr/lib/wsl/lib
/dev/loop7      222G  176G   46G   80% /mnt/c
```

VERSÕES DO OLLAMA

```
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ lsblk
NAME
MAJ:MIN RM  SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  256G  0 disk
sdb   8:16   0  256G  0 disk /
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ nvidia-smi
Command 'nvidia-smi' not found, but can be installed with:
sudo apt install nvidia-utils-525      # version 525.147.05-0
sudo apt install nvidia-utils-525-server # version 525.147.05-0
sudo apt install nvidia-utils-470      # version 470.256.02-0
sudo apt install nvidia-utils-470-server # version 470.256.02-0
sudo apt install nvidia-utils-535      # version 535.309.01-0
sudo apt install nvidia-utils-535-server # version 535.309.01-0
sudo apt install nvidia-utils-580      # version 580.159.03-0
```

Experimente a nova plataforma cruzada PowerShell <https://>

```
PS C:\Users\Enzom> wsl
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ ollama --version
ollama version is 0.34.0
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ ollama list
NAME      ID      SIZE      MODIFIED
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$ ollama serve
Error: listen tcp 127.0.0.1:11434: bind: address already
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/Enzom$
```

No ambiente utilizado, o Ollama identificou a execução por CPU e registrou 0 B de VRAM, caracterizando a utilização da CPU como recurso de processamento do modelo nesse ambiente.

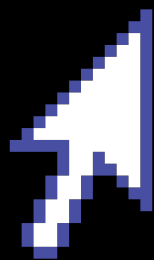
Para iniciar o servidor responsável pelo gerenciamento e execução dos modelos locais, foi utilizado o comando `ollama serve`, onde esse processo disponibiliza a API local do Ollama para que os comandos e aplicações possam se comunicar com o runtime.



PRIMEIRA EXECUÇÃO

Após a disponibilização do modelo, foi realizada uma execução inicial através do terminal. O objetivo dessa etapa é verificar se o Ollama consegue carregar o modelo e realizar uma inferência local. Como exemplo introdutório, viemos com um prompt simples e pergutamos ao modelo gemma-4-E2B-it-qat-q4_0-gguf:

“Responda em português: o que é um sistema operacional? Use apenas uma frase.”



Thinking Process:

1. ****Analyze the Request:**** The user wants to know "o que é um sistema operacional?" (what is an operating system?) and specifically asked for the answer in Portuguese. The crucial constraint is: "Use apenas uma frase" (Use only one sentence).
2. ****Identify Key Concepts:****
 - * System Software (Software fundamental).
 - * Manages hardware resources (Gerencia recursos de hardware).
 - * Acts as an interface between hardware and applications (Interface entre hardware e aplicações).
 - * Core function (Orquestrar e controlar).
3. ****Draft Potential Sentences (Focusing on conciseness and comprehensiveness):****
 - * ***Option 1 (Basic):*** Um sistema operacional é um software que gerencia os recursos do computador e permite a execução de outros programas.
 - * ***Option 2 (More detailed):*** É o software fundamental que atua como intermediário entre o hardware do computador e os softwares aplicativos, controlando e coordenando todas as operações.
4. ****Select the Best Single Sentence:**** Option 1 is clear, direct, and fits the "one sentence" constraint perfectly while covering the main function.
5. ****Final Review:****
 - * Language: Portuguese (Check).
 - * Content: Defines OS (Check).
 - * Length: One sentence (Check).
6. ****Output Generation.**** (Portuguese response)<channel|>Um sistema operacional é o software fundamental que gerencia os recursos de hardware e fornece a plataforma necessária para a execução de outros programas e aplicações.

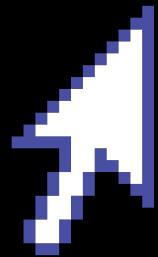
RESULTADOS

- Ambiente experimental preparado e funcional;
- Execução do Ollama + Gemma em ambiente Linux;
- Utilização de WSL2 e VirtualBox pela equipe;
- Modelo disponível para os experimentos posteriores;
- Base preparada para análise de RAM, contexto, quantização e desempenho;
- Open WebUI definido como interface gráfica, enquanto o Ollama realiza o processamento local.

```
6.  **Output Generation.** (Portuguese response)<channel|>Um sistema operacional é o software fundamental que gerencia os recursos de hardware e fornece a plataforma necessária para a execução de outros programas e aplicações.
```

```
enzo@DESKTOP-BHUFF6Q:/mnt/c/Users/EnzoM$
```

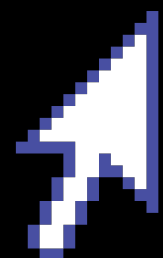
2- GIULIAN FABIO BASTOS AMORIM LIMA



POR QUE ESSA TRILHA É ADEQUADA?

1- A TRILHA A PERMITE ESTUDAR UMA APLICAÇÃO REAL COM CONTÊINER, SERVIÇO HTTP E PROCESSO DE INFERÊNCIA. OBSERVAMOS PORTAS, PERSISTÊNCIA, USO DE RAM E SWAP E ENCERRAMENTO POR FALTA DE MEMÓRIA.

2- ESSE AMBIENTE TAMBÉM PERMITE OS EXPERIMENTOS DA EQUIPE SOBRE PROCESSOS, THREADS, ESCALONAMENTO E CONCORRÊNCIA, RELACIONANDO A TEÓRIA DE SISTEMAS OPERACIONAIS AO COMPORTAMENTO DA APLICAÇÃO.



Item	Registro
Nome e finalidade	Open WebUI — interface web para selecionar modelos, conversar e manter histórico local.
Autoria e repositório	Timothy Jaeryang Baek; organização open-webui e colaboradores. https://github.com/open-webui/open-webui [1]
Licença	Open WebUI License, com condições de marca e contribuições sob licenças anteriores. A identidade visual foi preservada. [2]
Versão e acesso	v0.11.3 nos logs; imagem ghcr.io/open-webui/open-webui:main. Fontes acessadas em 10/09/2026. Commit Git não coletado.
Ambiente	Ubuntu 24.04.4 LTS no Oracle VirtualBox; 2 CPUs virtuais; última configuração: 3,8 GiB de RAM e 4 GiB de swap.
Dependências	Docker 29.1.3; Ollama 0.34.0. Aplicação com Python, FastAPI/Uvicorn, SvelteKit e SQLite. Versões internas não coletadas. [1]
Persistência	Volume open-webui em /app/backend/data, para banco SQLite, histórico e cache. Pesos do Ollama armazenados no Ubuntu. 

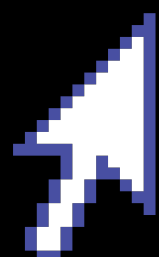
COMO A OPEN WEB CONVERSA COM OLLAMA VIA HOST.DOCKER.INTERNA?

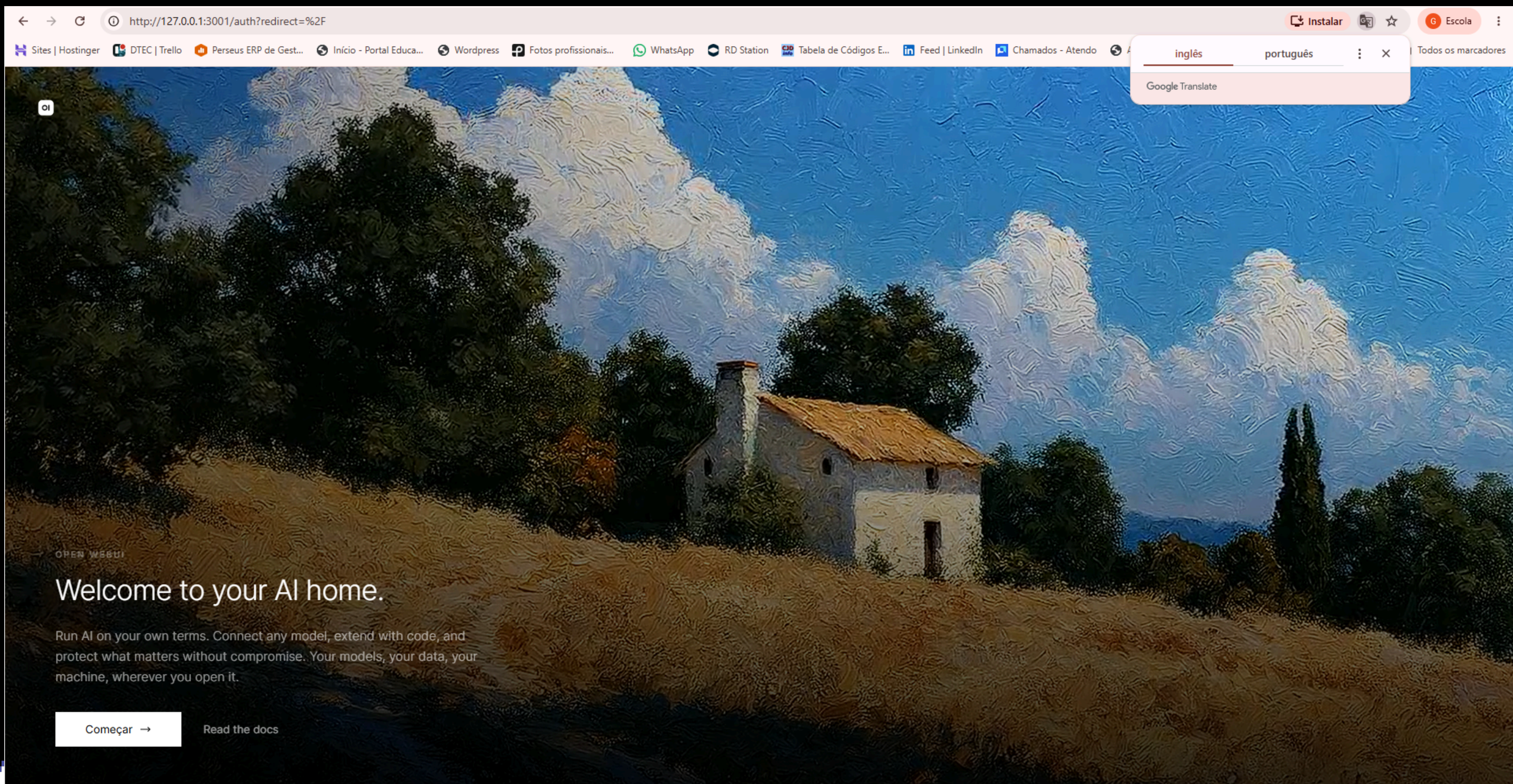
1- O OPEN WEBUI FOI EXECUTADO EM UM CONTÊNER DOCKER DENTRO DA MÁQUINA VIRTUAL

2- O OLLAMA FOI INSTALADO DIRETAMENTE NO UBUNTU.

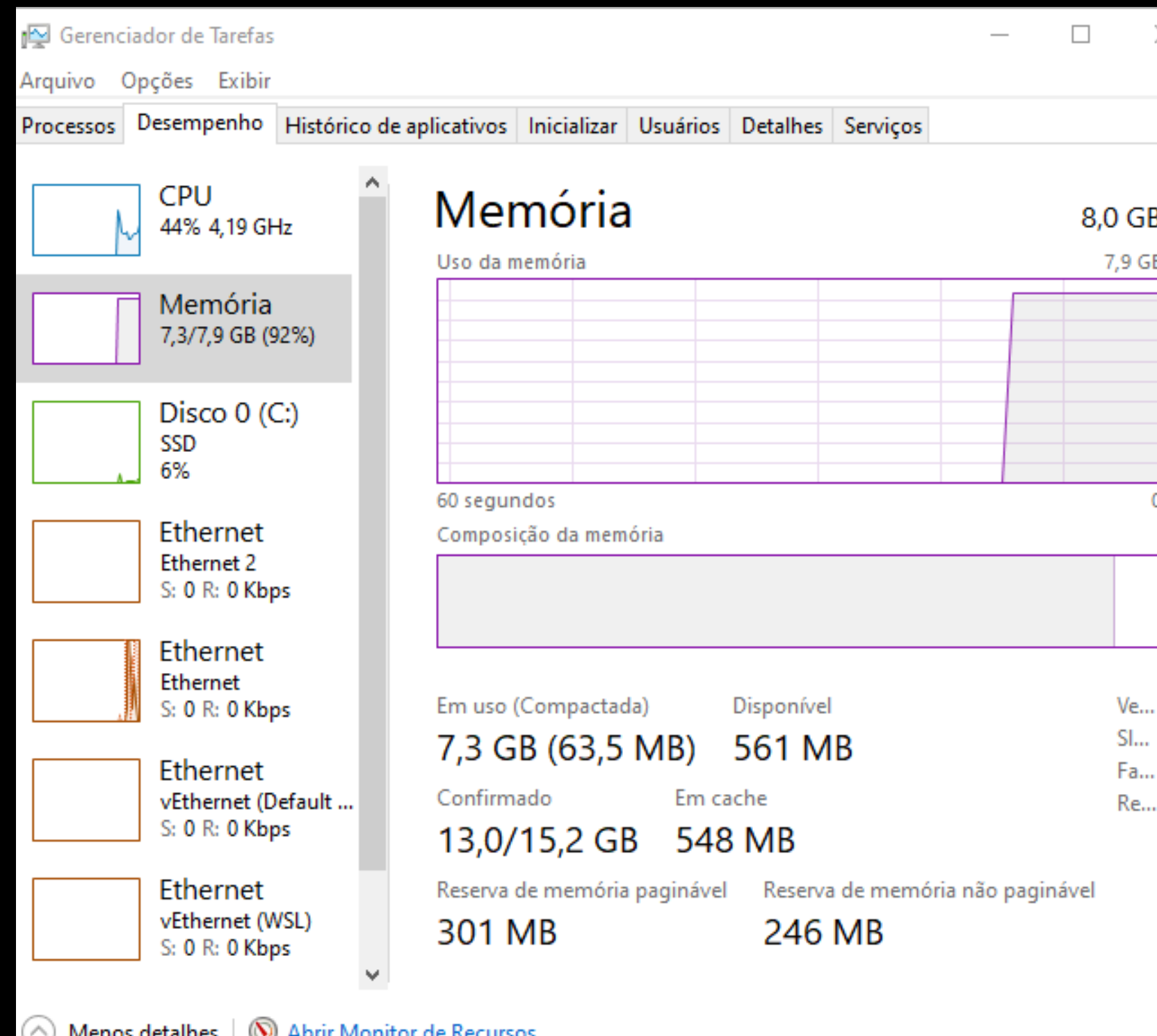
3 - O DOCKER FORNECE O AMBIENTE DA APLICAÇÃO E MANTÉM SEUS DADOS EM UM VOLUME PERSISTENTE.

4 - O OPEN WEBUI ENVIA AS MENSAGENS À API DO OLLAMA, QUE EXECUTA O GEMMA E DEVOLVE AS RESPOSTAS PARA EXIBIÇÃO NO NAVEGADOR.





ERROS ENCONTRADOS:



SOLUÇÃO PARA ERRO :

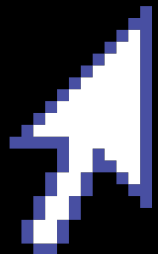
FOI ATIVADA SWAP DE 4 GIB. APÓS REDUÇÃO DE SERVIÇOS, A RAM DISPONÍVEL CHEGOU A 3,2 GIB.

COMANDO: SUDO FALLOCATE -L 4G /SWAPFILE
SUDO CHMOD 600 /SWAPFILE

```
vboxuser@so2:~$ free -h
```

	total	used	free	shared	buff/cache	available
Mem:	3.8Gi	663Mi	2.9Gi	344Ki	531Mi	3.2Gi
Swap:	4.0Gi	1.1Gi	2.9Gi			

```
vboxuser@so2:~$ sudo systemctl start ollama
```



ERROS ENCONTRADOS:

Responda em português: o que é um sistema operacional? Use apenas uma frase.

oi hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest

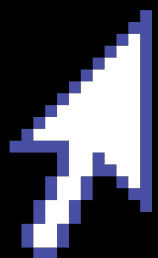
Pensou por um minuto ▾

O sistema operacional é o software essencial que gerencia os recursos de hardware e fornece a interface necessária para que os usuários e os programas interajam com o computador.

❗ {"error":{"code":400,"message":"request (5447 tokens) exceeds the available context size (4096 tokens), try increasing it","type":"exceed_context_size_error","n_prompt_tokens":5447,"n_ctx":4096}}

Oi

< 2/2 >



SOLUÇÃO PARA ERRO :

Chamada de Função

Padrão

Etiquetas de Raciocínio

Padrão

Semente

Padrão

Sequência de Paragem

Padrão

Temperatura

Padrão

Esforço de Raciocínio

Padrão

logit_bias

Padrão

max_tokens

Padrão

top_k

Padrão

top_p

Padrão



```
vboxuser@so2:~$ sudo systemctl start ollama
vboxuser@so2:~$ ollama run hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest --think=false "Responda em português, em uma frase: o que é um sistema operacional?"
Um sistema operacional é um software fundamental que gerencia os recursos do hardware de um computador e fornece uma plataforma para a execução de outros programas.

vboxuser@so2:~$
```

Responda em português: o que é um sistema operacional? Use apenas uma frase.

oi hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest

Pensou por um minuto ▾

O sistema operacional é o software essencial que gerencia os recursos de hardware e fornece a interface necessária para que os usuários e os programas interajam com o computador.

ⓘ {"error":{"code":400,"message":"request (5447 tokens) exceeds the available context size (4096 tokens), try increasing it","type":"exceed_context_size_error","n_prompt_tokens":5447,"n_ctx":4096}}

Oi

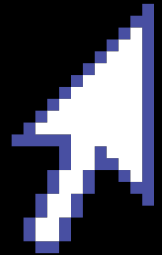
< 2/2 >

oi hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest

Olá! Como posso ajudar você hoje?

✎ 📄 🔊 ⓘ 👍 🗨️ 🌐 ↺ ↻

3 - RAUANY INGRID SANTOS DE JESUS





MODELO: GOOGLE/GEMMA-4-E2B-IT-QAT-Q4_0-GGUF

FICHA TÉCNICA

- 2,3B parâmetros efetivos (5,1B com embeddings)
 - 128K tokens de contexto
 - 3,35 GB (arquivo GGUF Q4_0)
 - Licença: Apache 2.0
-
- GGUF, derivado de checkpoint QAT (Quantization-Aware Training)
 - Cadeia: base → instruct ("-it") → QAT unquantized → conversão GGUF
 - Quantização Q4_0 já embutida no treino, não aplicada depois
-
- Ollama (via `ollama run hf.co/...`), llama.cpp, LM Studio, Jan, Unsloth Desktop



MODELO: GOOGLE/GEMMA-4-E2B-IT-QAT-Q4_0-GGUF



Por que essa escolha

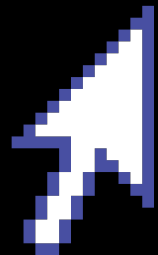
- Já vem em GGUF, pronto para Ollama — sem conversão extra
- QAT preserva qualidade próxima do BF16, mesmo em 4 bits
- Footprint leve (~2,3B efetivos) viável no hardware da equipe

Uso crítico de IA generativa — pontos verificados

- Confusão entre a Família do modelo
- Licença: conferida no repositório (Apache 2.0)
- Tamanho do arquivo: 3,35 GB a validar após download real (du -sh)
- Idioma: treino em 140+ idiomas não garante qualidade em português



4 - JOSÉ FAGNER SILVA JUNQUEIRA



PROCESSOS E THREADS

Seção 7.1 — observação do Ollama + Open WebUI

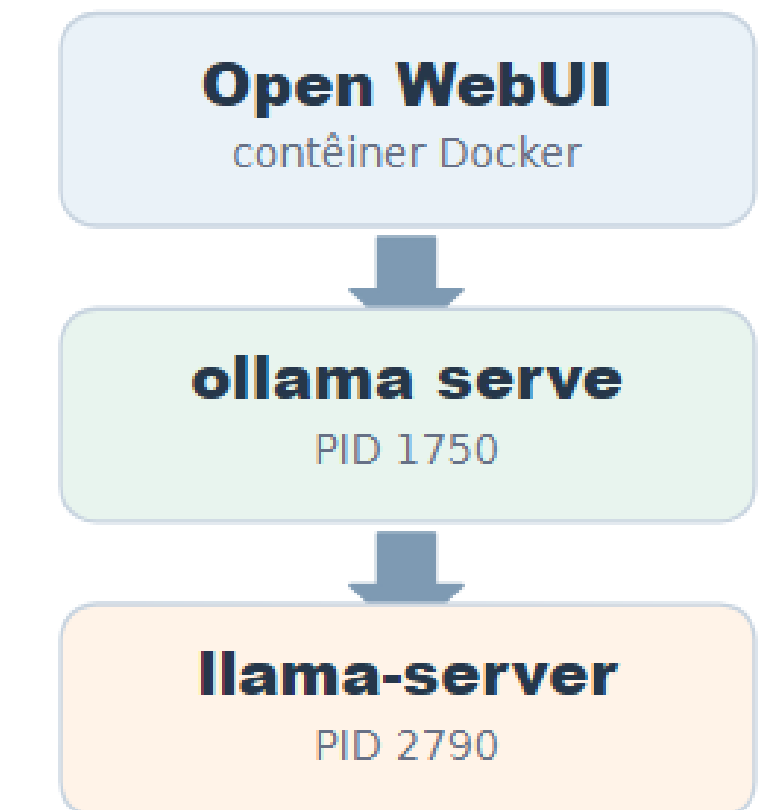
Objetivo: identificar processos principais e auxiliares, PID/PPID, estados, threads, uso de CPU/memória e relação de parentesco durante a inferência.

Ubuntu 26.04 / WSL2

Ollama 0.34.0

Open WebUI / Docker

Arquitetura observada



O QUE FOI OBSERVADO

Coleta realizada com ps, pstree, ps -L e top -H durante a inferência

Processos principais

ollama serve

PID 1750 • PPID 1

27 threads

Servidor / coordenação

llama-server

PID 2790 • PPID 1750

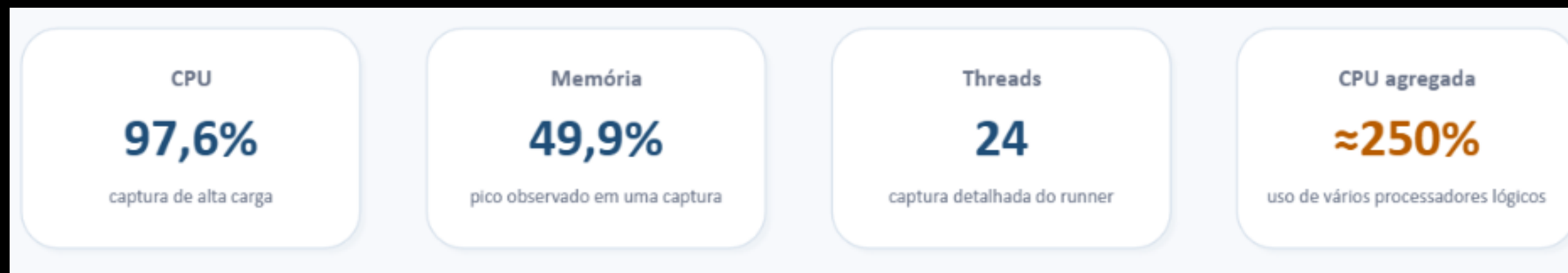
18–24 threads

Inferência do modelo

Evidências coletadas

- PID e PPID confirmaram a relação pai-filho entre Ollama e llama-server.
- STAT variou entre RI e SI conforme o instante da medição.
- NLWP mostrou múltiplas threads no runtime e no processo de inferência.
- ps -L revelou LWPs distintas e threads CUDA.
- top -H permitiu observar as threads individualmente.

COMPORTAMENTO DURANTE A INFERÊNCIA

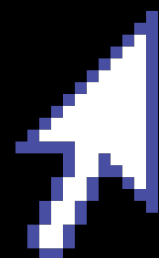


Interpretação

O ollama serve permanece como serviço principal e gerenciador das requisições.

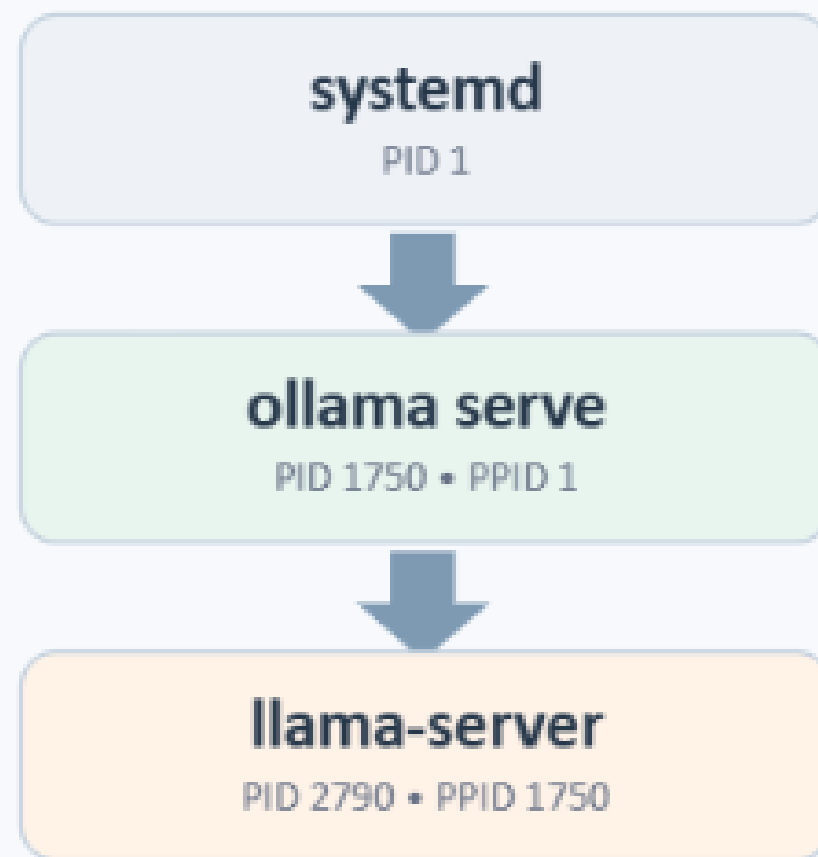
● O llama-server é o processo de trabalho pesado, criado/gerenciado pelo Ollama para executar o modelo.

● Valores acima de 100% de CPU são compatíveis com execução multithread em vários processadores lógicos.

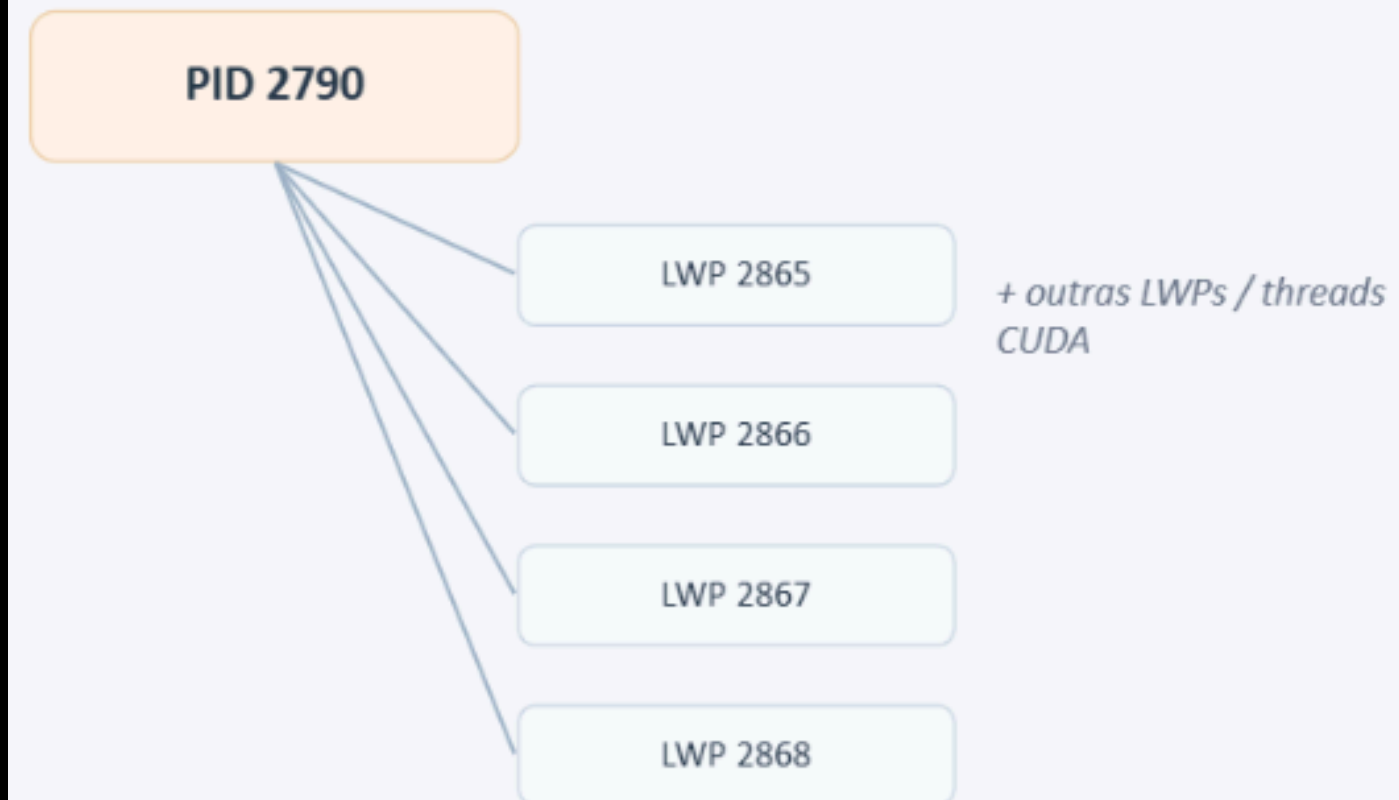


PARENTESCO E THREADS

Hierarquia de processos



Threads do llama-server



Mesmo PID de processo, identificadores LWP diferentes ⇒ execução multithread.

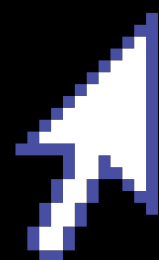
CONCLUSÕES – Q4 E Q5

Q4 — Processos e threads observados

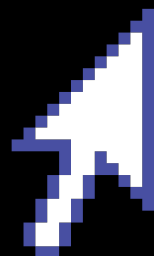
- ollama serve: PID 1750, PPID 1, 27 threads.
- llama-server: PID 2790, PPID 1750, 18–24 threads.
- Durante a inferência, o llama-server concentrou CPU e memória.
- Threads CUDA confirmaram uso de aceleração por GPU.
- Open WebUI permaneceu separado em contêiner Docker.

Q5 — Parentesco e responsabilidade

- systemd (PID 1) gerencia o serviço Ollama.
- ollama serve é o pai direto do llama-server.
- Ollama recebe e coordena as requisições.
- llama-server executa a inferência multithread do modelo.
- Open WebUI se comunica com o Ollama via HTTP; não é processo filho dele.



5 - FILIPE DE CARVALHO



CHAMADAS DE SISTEMA

Como o Ollama usa o sistema operacional para carregar um modelo de 4,3 GB e responder a um prompt

```
$ strace -f -c -o resumo.txt ollama run "$MODELO" "..."
```

UBUNTU 24.04.4
kernel 6.8.0-134

4 vCPUs
VirtualBox

7,8 GiB RAM
sem swap

SEM GPU
inferência em CPU

O COMANDO RASTREADO NÃO FAZ O TRABALHO

O ollama run é apenas um cliente HTTP.

ollama run
CLIENTE

11.866

chamadas de sistema

381

chamadas read

5,5 s

tempo em syscalls

ollama serve
DAEMON

54.465

chamadas de sistema

16.445

chamadas read

1.241 s

tempo em syscalls

94,72% do tempo do cliente é espera passiva — futex, epoll_pwait e nanosleep.

O MMAP NÃO MAPEIA OS PESOS

Das 214 chamadas, nenhuma mapeia o arquivo.

```
mmap(NULL, 2616033280, PROT_READ|PROT_WRITE,  
      MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)  
  
mmap(NULL, 1047973888, PROT_READ|PROT_WRITE,  
      MAP_PRIVATE|MAP_ANONYMOUS, -1, 0)
```

- **MAP_ANONYMOUS** mapeamento sem arquivo associado
- **-1** a posição do file descriptor — com arquivo, receberia o fd

OS PESOS ENTRAM POR READ, NÃO POR MAPEAMENTO

Abre o arquivo (openat) → Reserva um bloco vazio de memória (mmap)

→ Copia os dados do disco para dentro dele (read) → Pula entre tensores conforme precisa (lseek).

BYTES LIDOS DO DISCO

fd 5
5.224 chamadas **3,43 GiB**

fd 4
6.239 chamadas **0,95 GiB**

TOTAL

4,38 GiB
o ollama list declara 4,3 GB

AS QUATRO FAMÍLIAS DE CHAMADAS

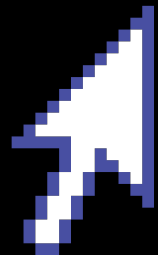
Resposta à Q6

CARGA	<code>openat · mmap · munmap</code>	5 blobs abertos, ~4,7 GiB de espaço de endereçamento reservado
DADOS	<code>read · pread64 · lseek</code>	4,38 GiB em 11.463 chamadas; maior leitura de 1,79 GiB
COMUNICAÇÃO	<code>socket · connect · accept4</code>	tudo em loopback - nada sai da máquina
LOGS	<code>write em stdout/stderr</code>	sem arquivo próprio; registro delegado ao systemd

FUTEX 87,44%

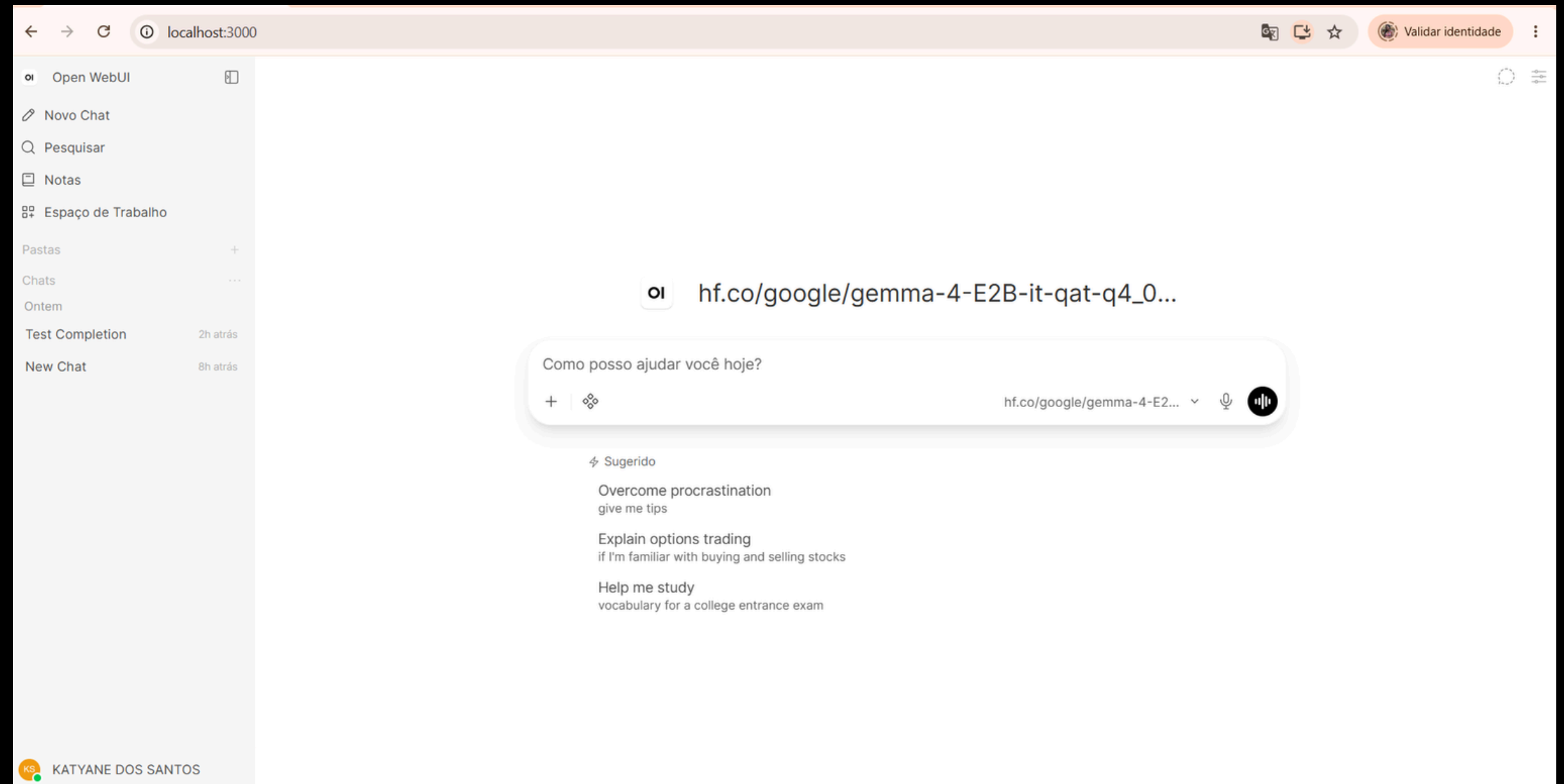
87% do tempo o programa está com threads esperando umas às outras. Com 13 threads brigando por 4 CPUs, o gargalo é coordenação, não disco.

6- KATYANE DOS SANTOS



EXPERIMENTOS – CONFIGURAÇÕES C1 E C2

Avaliação do impacto da concorrência no desempenho e no consumo de recursos do Gemma 4 executado localmente com Ollama.



AMBIENTE E MODELO UTILIZADO

```
WEDOT_BUILD_VERSION=08/c15852105001505/55e83181007027e300944
root@Katyane:/mnt/c/WINDOWS/system32# free -h
              total        used        free      shared  buff/cache   available
Mem:           4.8Gi        4.0Gi        108Mi        2.8Mi        842Mi        790Mi
Swap:          4.0Gi        2.3Gi        1.7Gi
root@Katyane:/mnt/c/WINDOWS/system32# ollama ps
NAME                                ID                SIZE      PROCESSOR  CONTEXT  UNTIL
hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest  bf543bb2aa66     3.9 GB    100% CPU   4096     4 minutes from now
root@Katyane:/mnt/c/WINDOWS/system32# ollama stop hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest
root@Katyane:/mnt/c/WINDOWS/system32# ollama ps
NAME    ID    SIZE  PROCESSOR  CONTEXT  UNTIL
root@Katyane:/mnt/c/WINDOWS/system32# /usr/bin/time -f "\nTEMPO_REAL=%e s\nCPU=%P\nMEM_MAX=%M KB" \
curl -s http://127.0.0.1:11434/api/generate \
  -d '{
```

-
-
- Modelo Gemma 4 E2B QAT Q4_0 executado localmente com Ollama no WSL2.
-
-

CONFIGURAÇÃO C1: EXECUÇÃO PADRÃO

```
root@Katyane: /mnt/c/WINDOWS/system32
EVAL_DURATION_NS=5473899000
LOAD_DURATION_NS=68333116842
root@Katyane:/mnt/c/WINDOWS/system32# ollama stop hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest
root@Katyane:/mnt/c/WINDOWS/system32#
root@Katyane:/mnt/c/WINDOWS/system32# python3 - <<'PY'
import urllib.request, json, time

model = "hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest"

prompt = """Explique detalhadamente como um sistema operacional gerencia os recursos de um computador.
Na resposta, aborde os seguintes aspectos: gerenciamento de processos, escalonamento da CPU,
threads, memória principal, memória virtual, sistema de arquivos, dispositivos de entrada e saída,
interrupções, chamadas de sistema e mecanismos de proteção e segurança.
Explique também como esses componentes se relacionam durante a execução simultânea de vários
programas. Organize a explicação de maneira didática, utilizando aproximadamente 10 parágrafos."""

dados = json.dumps({
    "model": model,
    "prompt": prompt,
    "stream": True,
    "think": False
}).encode("utf-8")

req = urllib.request.Request(
    "http://127.0.0.1:11434/api/generate",
    data=dados,
    headers={"Content-Type": "application/json"}
)

inicio = time.perf_counter()
ttft = None
final = None

with urllib.request.urlopen(req, timeout=600) as resposta:
    for linha in resposta:
        obj = json.loads(linha.decode("utf-8"))

        if ttft is None and obj.get("response"):
            ttft = time.perf_counter() - inicio

        if obj.get("done"):
            final = obj

total = time.perf_counter() - inicio

eval_count = final.get("eval_count", 0)
eval_duration = final.get("eval_duration", 0)

tokens_s = eval_count / (eval_duration / 1e9) if eval_duration else 0
```

```
print(f"TTFT={ttft:.3f} s")
print(f"TEMPO_TOTAL={total:.3f} s")
print(f"PROMPT_EVAL_COUNT={final.get('prompt_eval_count')}")
print(f"EVAL_COUNT={eval_count}")
print(f"EVAL_DURATION_NS={eval_duration}")
print(f"LOAD_DURATION_NS={final.get('load_duration')}")
print(f"TOKENS_POR_SEGUNDO={tokens_s:.3f}")
PY
TTFT=84.370 s
TEMPO_TOTAL=156.704 s
PROMPT_EVAL_COUNT=120
EVAL_COUNT=1132
EVAL_DURATION_NS=71349992000
LOAD_DURATION_NS=82442139144
TOKENS_POR_SEGUNDO=15.865
```

Evidência da execução individual utilizada na configuração C1.

CONFIGURAÇÃO C2: 3 REQUISIÇÕES SIMULTÂNEAS

```
root@Katyanē:/mnt/c/WINDOWS/system32# ollama stop hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest
root@Katyanē:/mnt/c/WINDOWS/system32# python3 /tmp/c2_grande.py
```

REQUISICAO 1

TTFT=200.041 s

TEMPO_TOTAL=267.415 s

PROMPT_EVAL_COUNT=120

EVAL_COUNT=1086

EVAL_DURATION_NS=67132569000

LOAD_DURATION_NS=69427638239

TOKENS_POR_SEGUNDO=16.177

REQUISICAO 2

TTFT=133.222 s

TEMPO_TOTAL=199.869 s

PROMPT_EVAL_COUNT=120

EVAL_COUNT=1122

EVAL_DURATION_NS=66418969000

LOAD_DURATION_NS=69424734409

TOKENS_POR_SEGUNDO=16.893

REQUISICAO 3

TTFT=72.311 s

TEMPO_TOTAL=133.038 s

PROMPT_EVAL_COUNT=120

EVAL_COUNT=1001

EVAL_DURATION_NS=60216056000

LOAD_DURATION_NS=69429244335

TOKENS_POR_SEGUNDO=16.623

A concorrência provocou espera entre as requisições na carga mais pesada.

CPU, RAM E THREADS - MONITORAMENTO DE RECURSOS – C2 GRANDE

```
2026-09-12T23:54:52 PID=3486 CPU=273.0% RAM=3426.6 MB THREADS=15
2026-09-12T23:54:53 PID=3486 CPU=274.0% RAM=3426.6 MB THREADS=15
2026-09-12T23:54:54 PID=3486 CPU=273.0% RAM=3426.6 MB THREADS=15
2026-09-12T23:54:55 PID=3486 CPU=272.0% RAM=3426.6 MB THREADS=15
^C
Monitor encerrado.
root@Katyane:/mnt/c/WINDOWS/system32# python3 - <<'PY'
import csv

with open("/tmp/monitor_ollama.csv") as f:
    dados = list(csv.DictReader(f))

cpu = [float(x["cpu_percent"]) for x in dados]
ram = [int(x["rss_kb"]) / 1024 for x in dados]
threads = [int(x["threads"]) for x in dados]

print(f"AMOSTRAS={len(dados)}")
print(f"CPU_MEDIA={sum(cpu)/len(cpu):.2f}%")
print(f"CPU_PICO={max(cpu):.2f}%")
print(f"RAM_MEDIA={sum(ram)/len(ram):.2f} MB")
print(f"RAM_PICO={max(ram):.2f} MB")
print(f"THREADS_MEDIA={sum(threads)/len(threads):.2f}")
print(f"THREADS_MAX={max(threads)}")
PY
AMOSTRAS=282
CPU_MEDIA=231.00%
CPU_PICO=310.00%
RAM_MEDIA=3387.51 MB
RAM_PICO=3938.38 MB
THREADS_MEDIA=14.49
THREADS_MAX=15
```

A CPU foi o recurso que apresentou o maior aumento com a concorrência

COMPARAÇÃO C1 * C2

```
> root@Katyane: /mnt/c/WINDOWS/system32
```

COMPARAÇÃO DE DESEMPENHO – C1 x C2

TTFT MÉDIO – CARGA GRANDE

C1			89,57 s
C2			158,33 s

CPU MÉDIA – CARGA GRANDE

C1			150,31%
C2			231,00%

RESULTADO:

A concorrência aumentou o TTFT em aproximadamente 76,8%.

C1 = 1 requisição por vez

C2 = 3 requisições simultâneas

CONCLUSÃO

```
root@Katyane: /mnt/c/WINDOWS/system32

=====
CONCLUSÃO – CONFIGURAÇÕES C1 x C2
=====

RESPONSIVIDADE
-----
Carga pequena -> Pouca diferença entre C1 e C2
Carga grande -> TTFT aumentou com a concorrência

TTFT carga grande:
C1: 89,57 s -> C2: 158,33 s (+76,8%)

=====
RECURSOS COMPUTACIONAIS
-----
CPU média:
C1: 150,31% -> C2: 231,00%

RAM média:
C1: 3,38 GB -> C2: 3,39 GB

Threads máximas:
C1: 15 -> C2: 15

=====
CONCLUSÃO
-----
A concorrência favorece o atendimento de múltiplas
solicitações, mas, sob cargas maiores, aumenta a competição
por CPU e a latência individual.

TRADE-OFF:
Throughput x Latência x Utilização de recursos
-----
```

AMBIENTE EXPERIMENTAL - C3

VIRTUALIZAÇÃO E SISTEMA

Oracle VirtualBox
Ubuntu 24.04.4 LTS

RECURSOS DA VM

AMD Ryzen 7 7435HS
2 processadores lógicos
3,8 GiB de RAM
4 GiB de swap
Execução em CPU

```
Memory size:          4096MB
Number of CPUs:        2
Configured memory balloon: 0MB
```

```
Mem:               total      used      free      shared  buff/cache  available
Swap:             4.0Gi       723Mi    3.3Gi
vboxuser@so2:~$ |

Name                                NumberOfCores  NumberOfLogicalProcessors
----                                -
AMD Ryzen 7 7435HS                  8              16
```

METODOLOGIA – CONFIGURAÇÃO C3

VARIÁVEL ALTERADA

1024 tokens × 4096 tokens

Janela de contexto

num_ctx

CONDIÇÕES MANTIDAS

Mesmo modelo

Mesmo prompt

Mesmo ambiente

Mesma quantização

Execução em CPU

```
curl -s http://127.0.0.1:11434/api/generate \
-d '{
  "model": "...",
  "prompt": "Explique em uma frase o que é um sistema operacional.",
  "think": false,
  "stream": false,
  "keep_alive": 0,
  "options": {
    "num_ctx": 4096
  }
}'
```

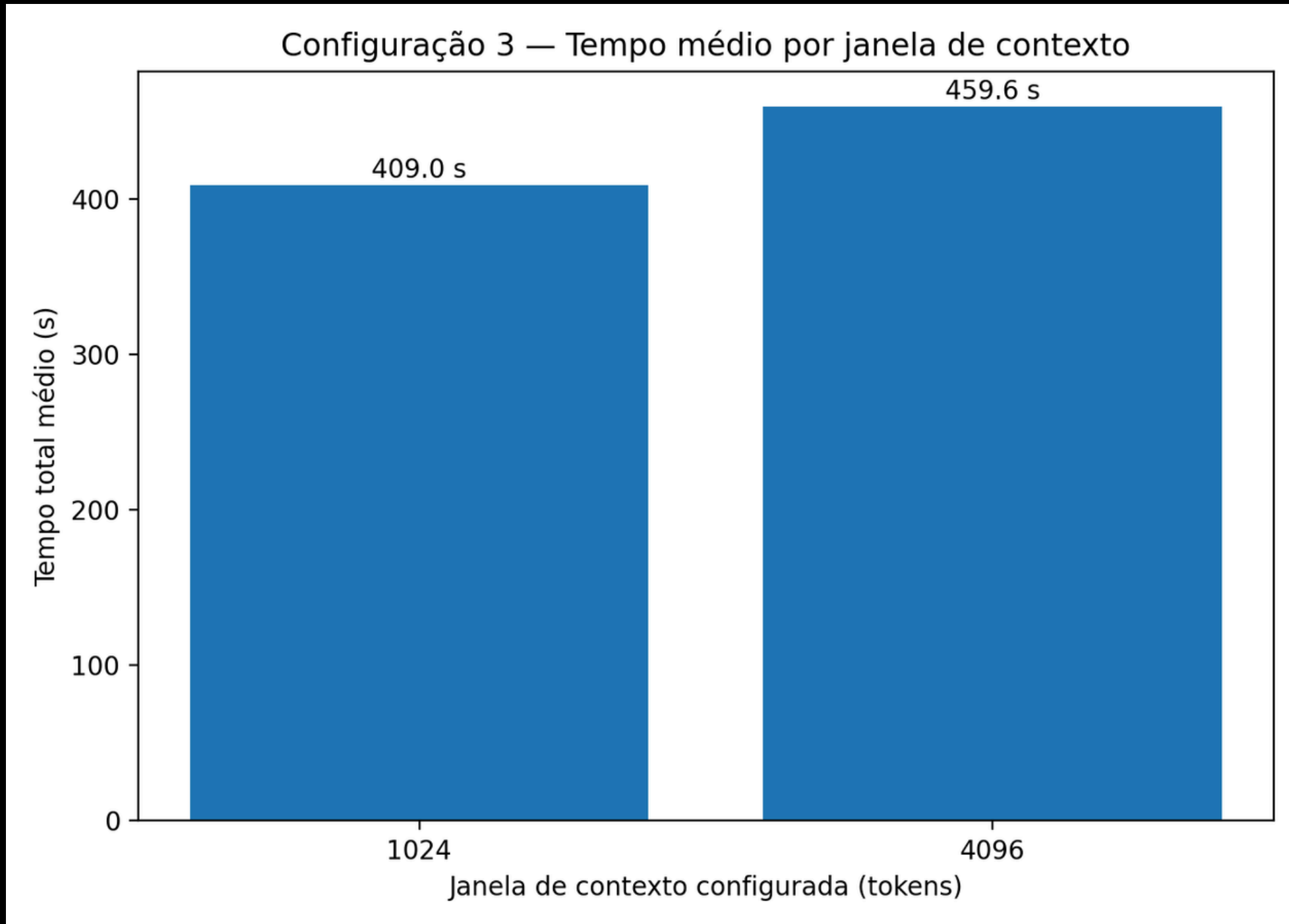
Exemplo da requisição com num_ctx = 4096

RESULTADOS – CONFIGURAÇÃO C3

Execução	Contexto	Resultado	Tempo total 
Curto 1	1024	Sucesso	392,6 s
Curto 2	1024	Sucesso	425,3 s
Longo 1	4096	Sucesso	511,9 s
Longo 2	4096	Timeout	—
Longo 3	4096	Sucesso	407,3 s

```
vboxuser@so2:~$ curl -s http://127.0.0.1:11434/api/generate \
-d '{
  "model": "hf.co/google/gemma-4-E2B-it-qat-q4_0-gguf:latest",
  "prompt": "Explique em uma frase o que é um sistema operacional.",
  "think": false,
  "stream": false,
  "keep_alive": 0,
  "options": {
    "num_ctx": 4096
  }
}' > longo2.json
vboxuser@so2:~$ python3 -m json.tool longo2.json
{
  "error": "timed out waiting for llama-server to start - "
```

COMPARAÇÃO DOS RESULTADOS – C3



+12,4%
tempo total médio
+127%
tempo médio de geração
-71,7%
velocidade média

LIMITAÇÕES E CONCLUSÃO - C3

LIMITAÇÕES

3,8 GiB de RAM na VM
4 GiB de swap necessários
OOM Killer sem swap
1 timeout com 4096 tokens

CONCLUSÃO

4096 tokens aumentou o
tempo total
Tempo de geração também
aumentou
Velocidade média caiu
Maior contexto reduziu a
previsibilidade

CONCLUSÃO DO TRABALHO

A execução local de IA é viável, mas seu desempenho depende diretamente dos recursos administrados pelo Sistema Operacional.

EXECUÇÃO LOCAL

Ollama

Open WebUI

Gemma

Funcionaram Localmente

RECURSOS DO SO

CPU, RAM, SWAP,
processos e threads
impactam desempenho
e estabilidade.

TRADE-OFFS

Mais concorrência →
maior latência

Maior contexto → maior
custo e menor
previsibilidade

OBRIGADO!